Dept. of Intelligent Mechatronics Engineering,
Dept. of Convergence for Intelligent Drone, Sejong University

BLOCK – BASED VALET PARKING MANAGEMENT SYSTEM

PRESENTER : JUN OH CHOI

ADVISE PROFESSOR : JIN WOO SONG

2024. 8. 21

BACKGROUND

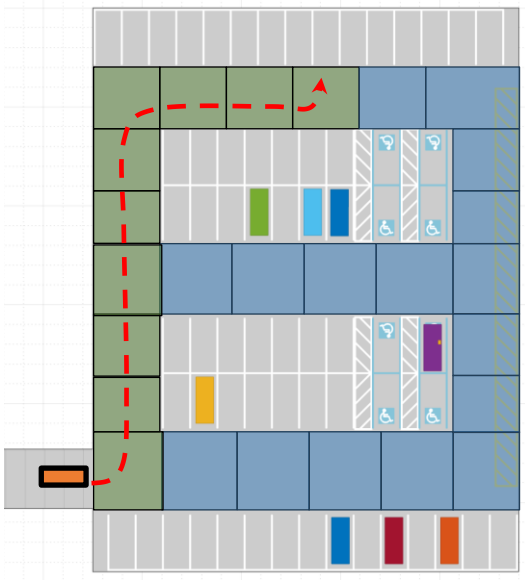
Mobility Parking Automation Service

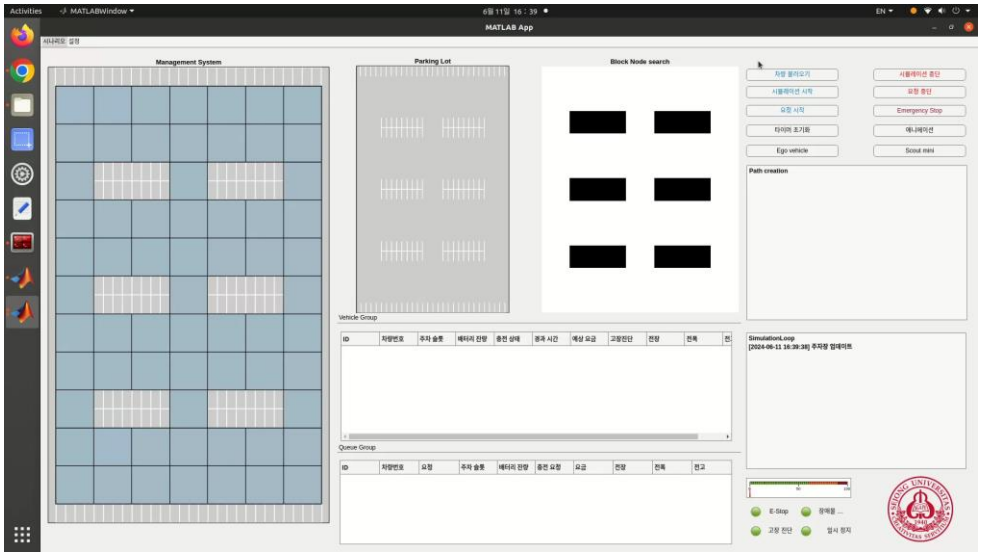
The Importance of developing parking lot technology

- In the case of Seoul, the parking space availability rate was 137.1% as of 2021
- There is a critical need to address the information asymmetry regarding parking availability.

Expected effects of technology development

- Technical aspect - Expansion of autonomous driving technology and acceleration of the autonomous driving era.
- Economic and Industrial aspect - Direct and indirect job creation through service development and operations, as well as revenue generation from the sale and export of parking lot systems.
- Social aspect - Reduction in parking-related traffic accidents.



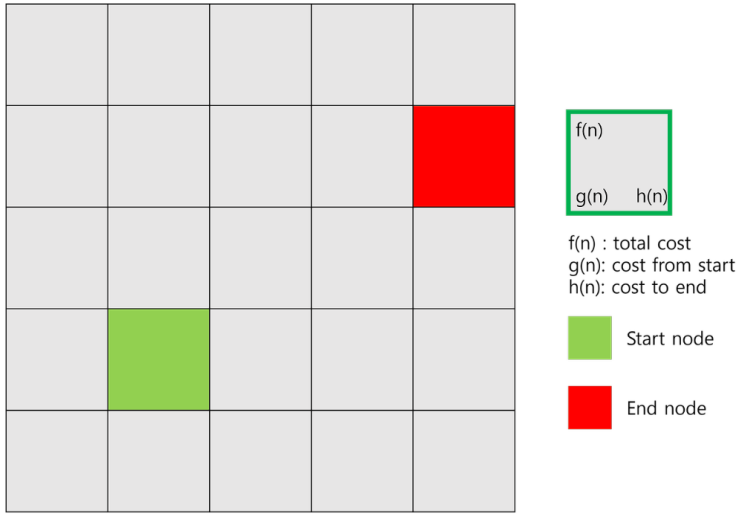


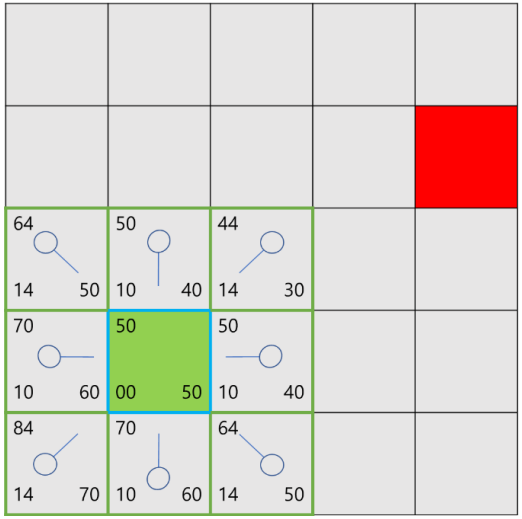
BASIC PRINCIPLE

- **The A* algorithm** is a representative algorithm for pathfinding
- While it is similar to Dijkstra's algorithm as a graph search algorithm, it differs in that it utilizes heuristic estimates

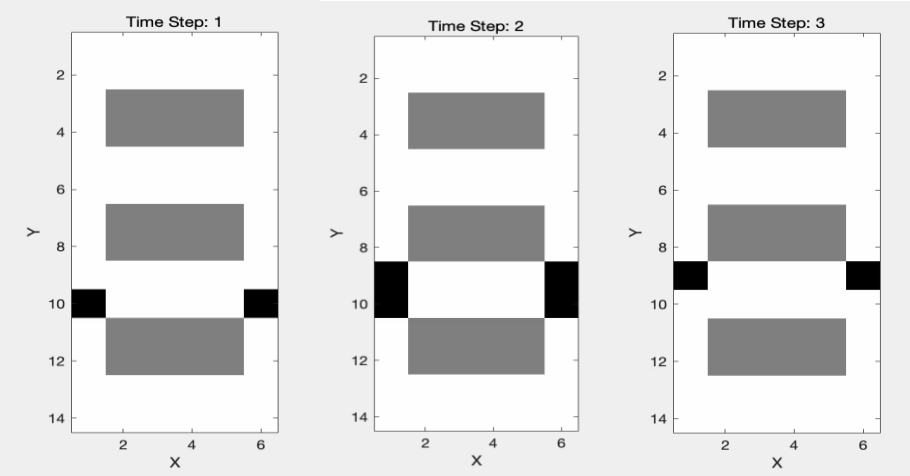
$$f(n) = g(n) + h(n)$$

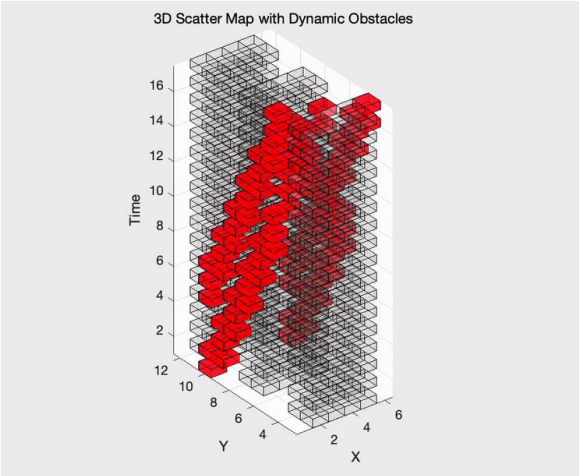
$$f(n) : total\ cost, g(n) : cost\ from\ start, h(n): cost\ to\ end$$

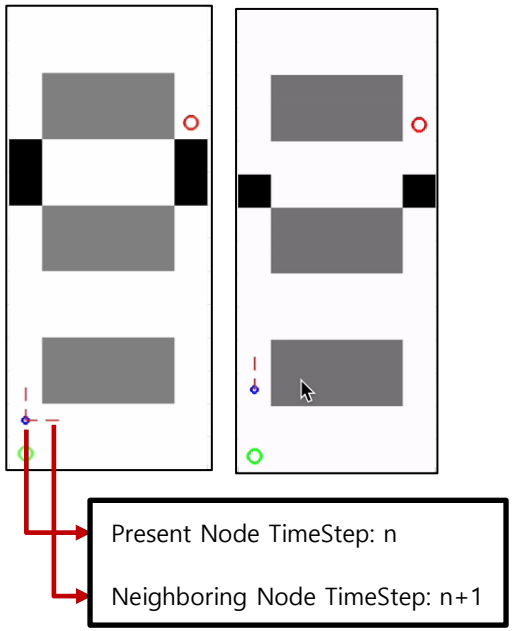




BLOCK - BASED PATH PLANNING ALGORITHM





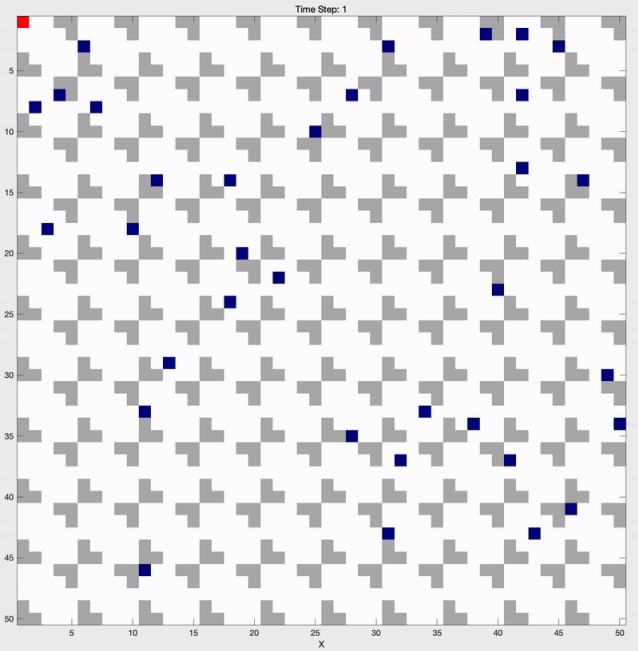


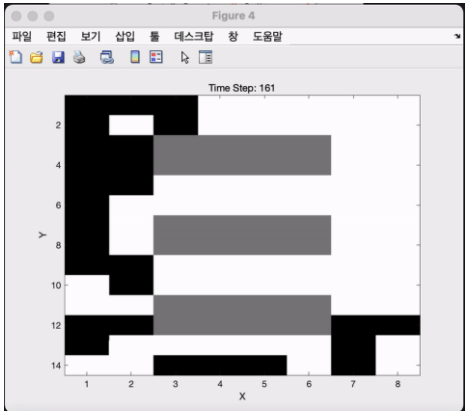
$$f(n) = h(n) + g(n) + t(n)$$

$t(n)$: *time cost from start*

- Extending a 2D Parking Lot grid map to 3D by adding a time dimension
- Pathfinding by minimizing the $f(n)$ value, which is the sum of the existing A* algorithm formula and $t(n)$

BLOCK - BASED PATH PLANNING ALGORITHM





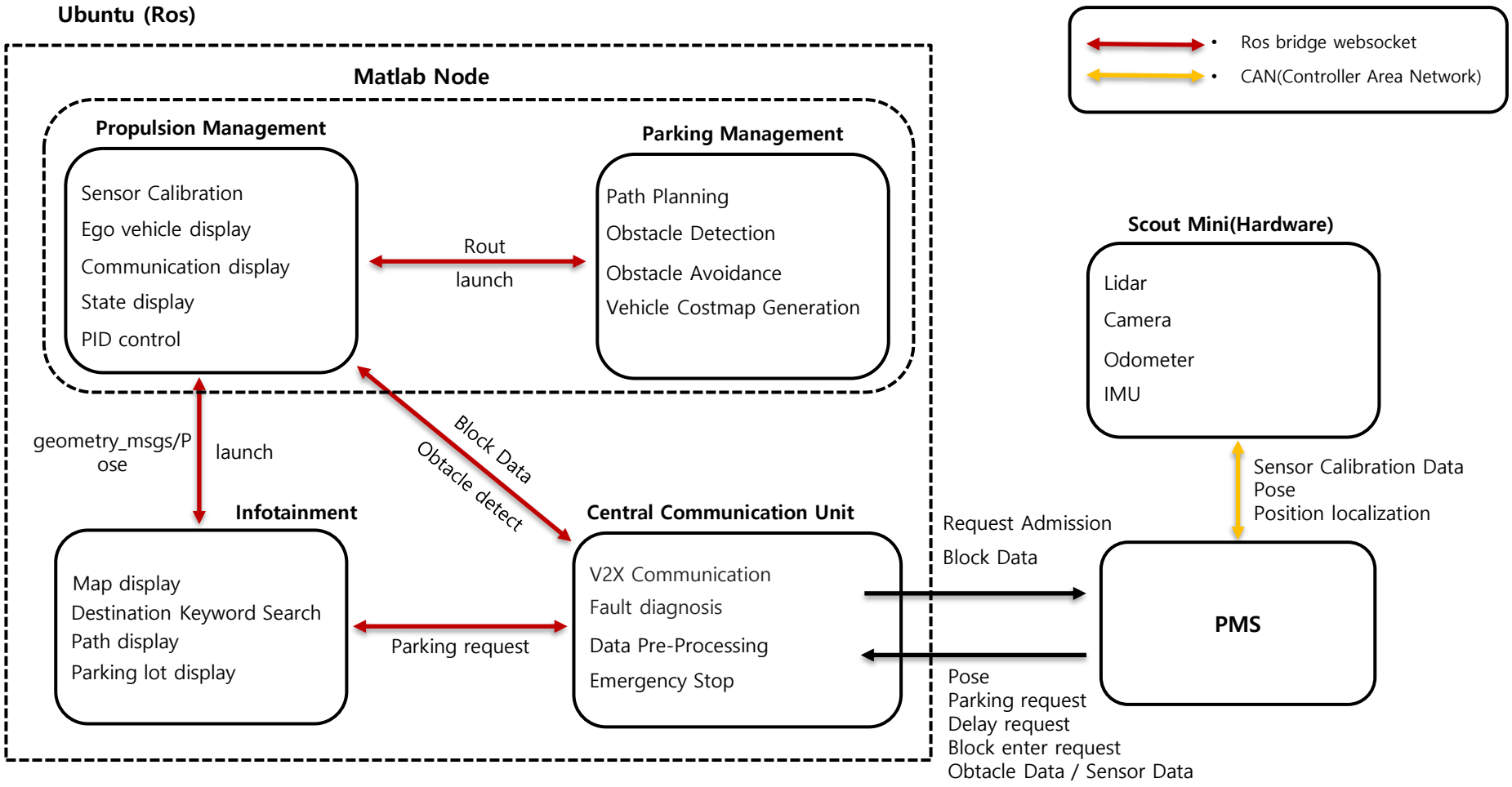

```
[출/입차 요청 차량: 100대, 입차: 35대, 출차: 65대]
[98] 시뮬레이션 성공! (총 탐색 노드: 9467, 소요 Step:194, 소요 시간: 0.090184 초)
[출/입차 요청 차량: 100대, 입차: 35대, 출차: 65대]
[99] 시뮬레이션 성공! (총 탐색 노드: 8105, 소요 Step:203, 소요 시간: 0.074632 초)
[출/입차 요청 차량: 100대, 입차: 45대, 출차: 55대]
[100] 시뮬레이션 성공! (총 탐색 노드: 9931, 소요 Step:189, 소요 시간: 0.089493 초)
평균 실행 시간: 0.090643 초
fx >>
```

Algorithm validation 1

- Validation Environment : 50 by 50 Map
- Path from top-left corner to the bottom-right corner
- Generate and navigate a path with minimal cost while considering obstacles to avoid collisions

Algorithm validation 2

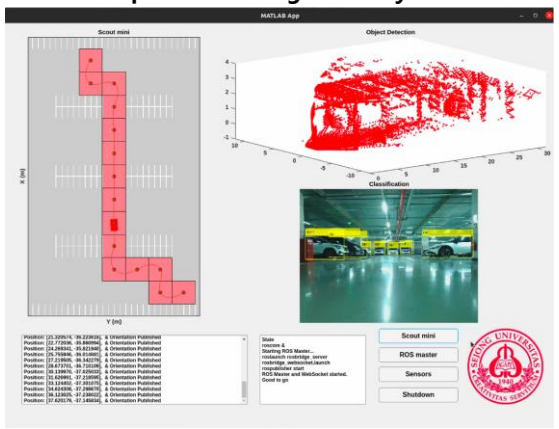
- Simultaneous commands for 100 vehicles to enter and 100 vehicles to exit.
- Verify that the vehicles navigate along optimal paths while minimizing empty spaces int the routes.
- The average time taken for 100 simulations was 0.091 seconds


Ubuntu (Ros)
Matlab Node
Propulsion Management
Sensor Calibration
Ego vehicle display
Communication display
State display
PID control
Parking Management
Path Planning
Obstacle Detection
Obstacle Avoidance
Vehicle Costmap Generation
Rout
launch
geometry_msgs/Pose
launch
Block Data
Obtacle detect
Infotainment
Map display
Destination Keyword Search
Path display
Parking lot display
Parking request
Central Communication Unit
V2X Communication
Fault diagnosis
Data Pre-Processing
Emergency Stop
Ros bridge websocket
CAN(Controller Area Network)
Scout Mini(Hardware)
Lidar
Camera
Odometer
IMU
Sensor Calibration Data
Pose
Position localization
Request Admission
Block Data
PMS
Pose
Parking request
Delay request
Block enter request
Obtacle Data / Sensor Data

Parking Management System



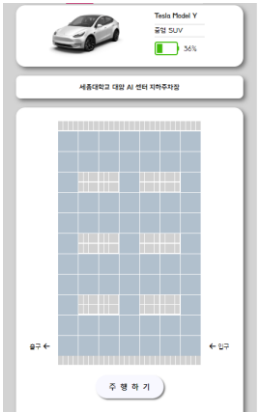
Propulsion Management System



PMS

Mobile Application





Scout mini



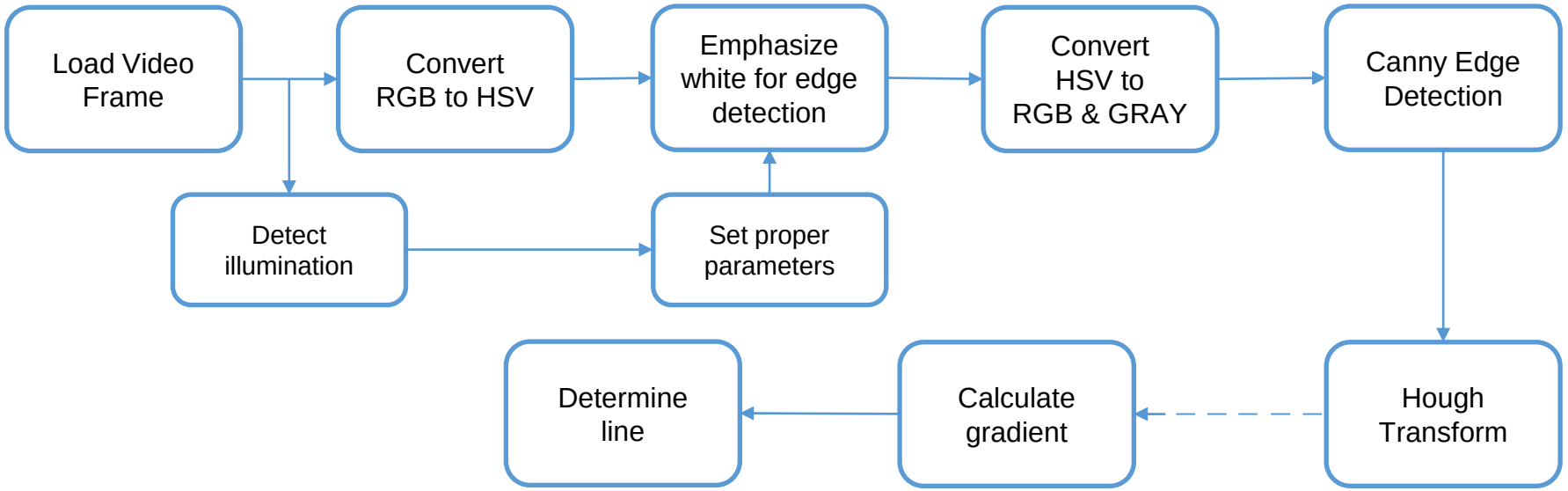
Q & A



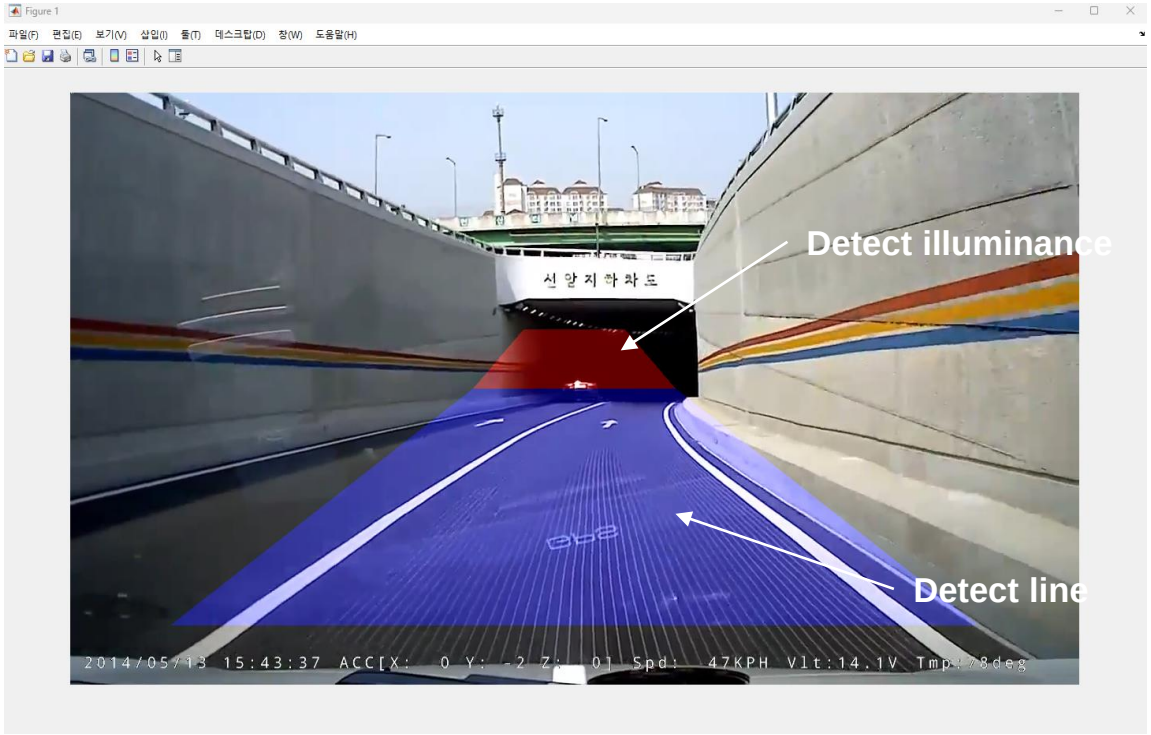
조도 변화에 강인한 차선 검출을 위한 명도 필터링 알고리즘 제안

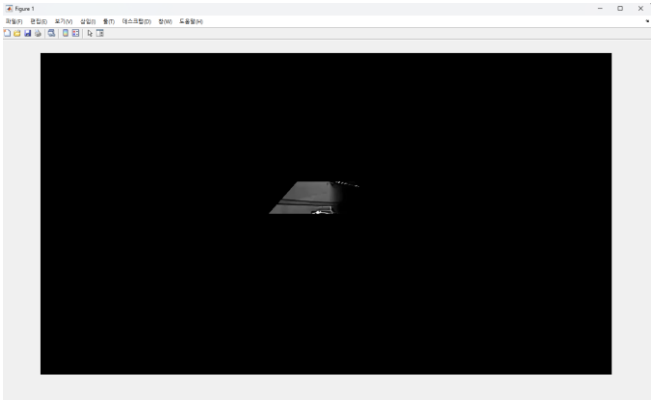
INCSL(지능형 항법 제어 시스템 연구실)
강어진

LINE DETECTION

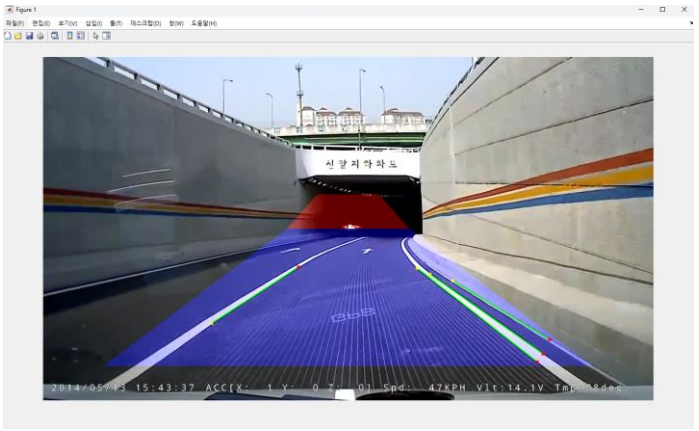


ROI, REGION OF INTEREST



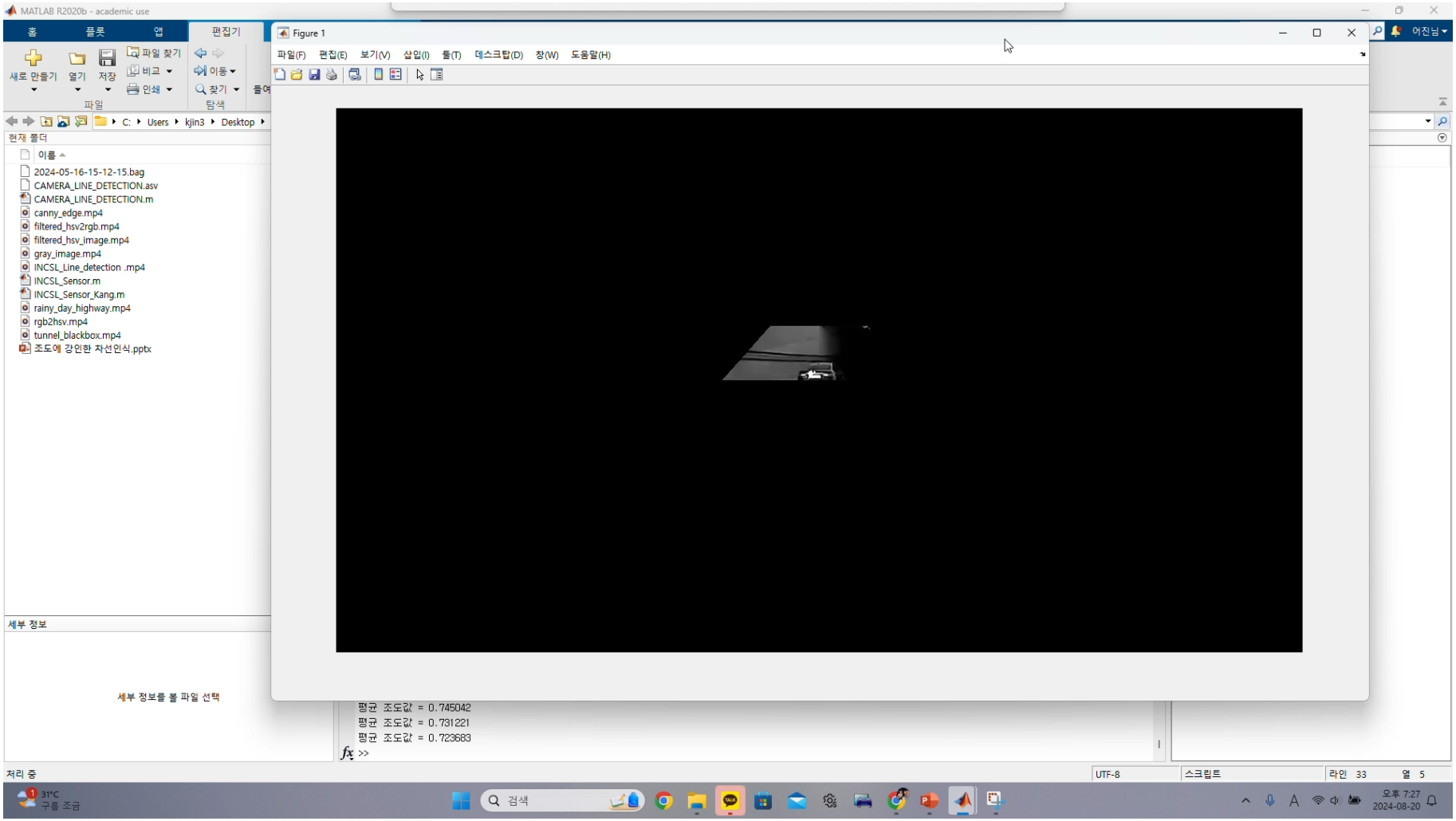


Mean of pixels



Canny Edge Detection
+ Hough Transform

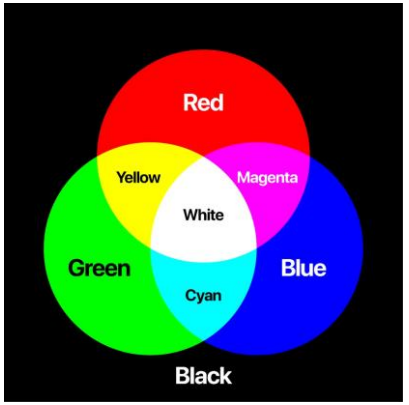
ROI, REGION OF INTEREST



RGB, HSV, GRAY SCALE

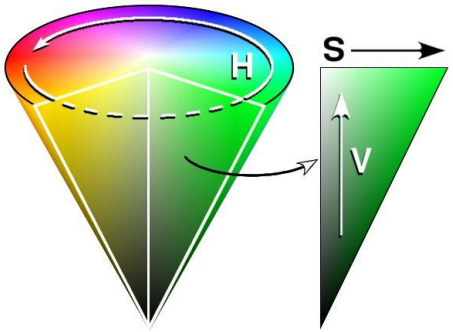
RGB

- Red, Green, Blue
- 0 ~ 255



HSV

- Hue(색상), Saturation(채도), Value(명도)
- Easily Emphasize white



- H : 0° ~ 360° (Color, 0° = Red)
- S : 0 ~ 100% (Depth of Color)
- V : 0 ~ 100% (Brightness for Color, 0% = Black)

Gray Scale

- Mean Gray = (R + G + B) / 3
- Gray = (0.299 * R) + (0.587 * G) + (0.114 * B) →Open CV (Y-Gray Scale)

```
% HSV 색 공간으로 변환
hsv_image = rgb2hsv(I);

% 흰색 범위 설정
hue_min = 0;        % Hue 범위 (흰색에서는 Hue 값이 중요하지 않음)
hue_max = 1;        % Hue 범위 (흰색에서는 Hue 값이 중요하지 않음)

sat_min = 0;        % 채도 범위 (흰색의 경우 낮은 채도/기본값 0)
sat_max = 0.1;      % 채도 범위 (기본값 0.3)

val_min = 0.7;      % 명도 범위 (흰색의 경우 높은 명도/기본값 0.7)
val_max = 1;        % 명도 범위 (기본값 1)

% 흰색 범위에 맞는 픽셀 검출
mask = (hsv_image(:,:,1) >= hue_min) & (hsv_image(:,:,1) <= hue_max) & ...
       (hsv_image(:,:,2) >= sat_min) & (hsv_image(:,:,2) <= sat_max) & ...
       (hsv_image(:,:,3) >= val_min) & (hsv_image(:,:,3) <= val_max);

% 마스크를 사용하여 흰색 차선만 추출, 원래 이미지에서 흰색만 표시
filtered_image = bsxfun(@times, hsv_image, cast(mask, 'like', hsv_image));

filtered_image = hsv2rgb(filtered_image);
```

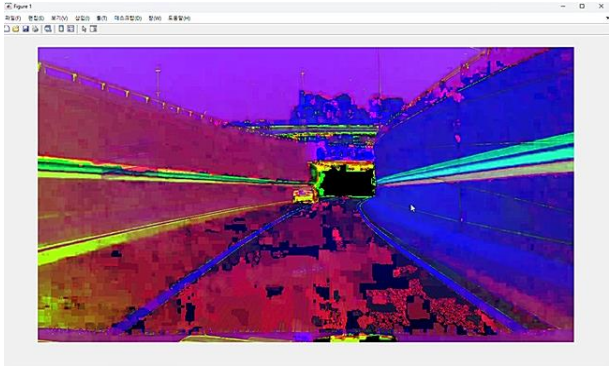
RGB, HSV, GRAY SCALE



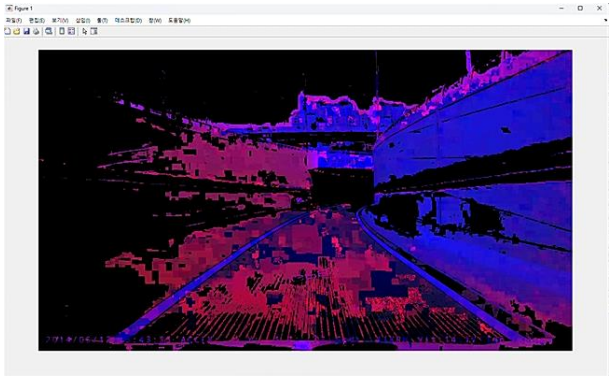
Original video
(RGB)



HSV→RGB



RGB → HSV



Filtered HSV



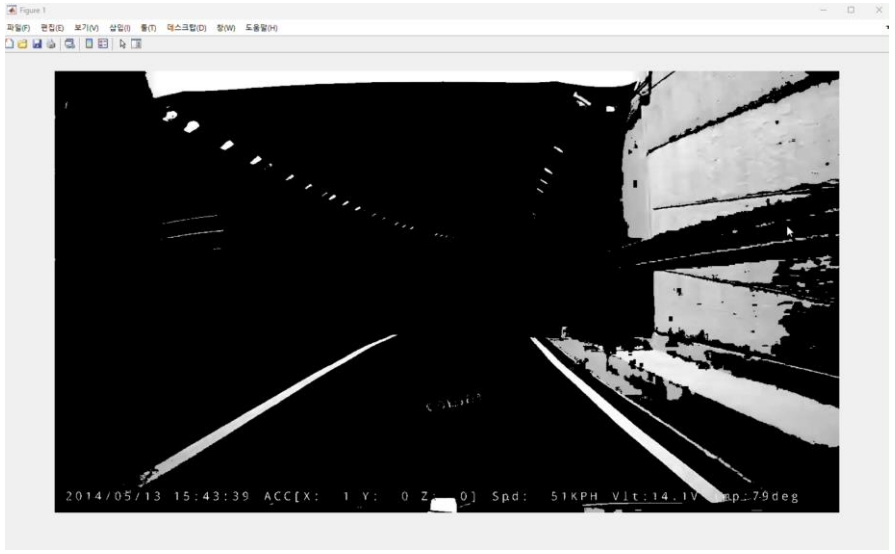
Gray Scale

RGB, HSV, GRAY SCALE

명도 0.3 ~ 1.0 필터링



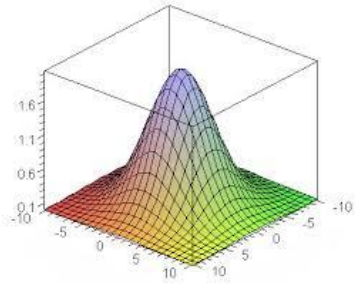
명도 0.7 ~ 1.0 필터링



CANNY EDGE DETECTION

- 1. Gaussain blurring

  가우시안 함수를 이용하여
  각 픽셀의 값을 주변 픽셀들의
  가중 평균값으로 변경. (blur 처리)





- 2. Detect direction and intensity (Sobel mask)

수평 $G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$ 수직 $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$

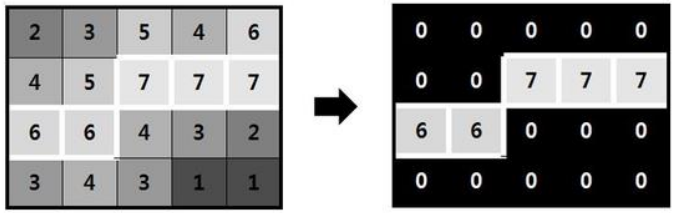
3x3 Convolution

$$Edge_Gradient\,(G) = \sqrt{G_x^2 + G_y^2}$$

$$Angle(\theta) = \tan^{-1}\left(\frac{G_x}{G_y}\right)$$



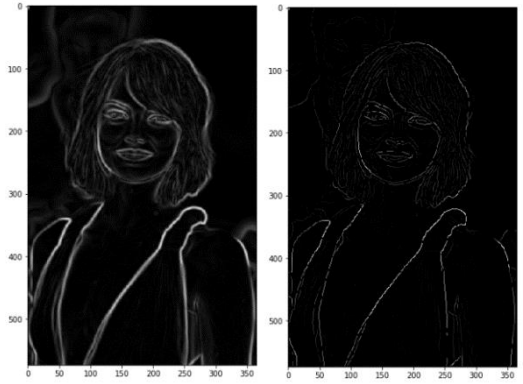
CANNY EDGE DETECTION



- 3. Non-maximum suppression (NMS)

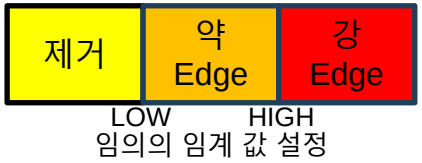
  -픽셀을 얇게 만듦. (1픽셀 단위)

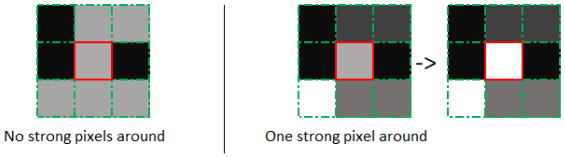
  -뭉개진 직선을 본래 직선으로 만들어주는 역할

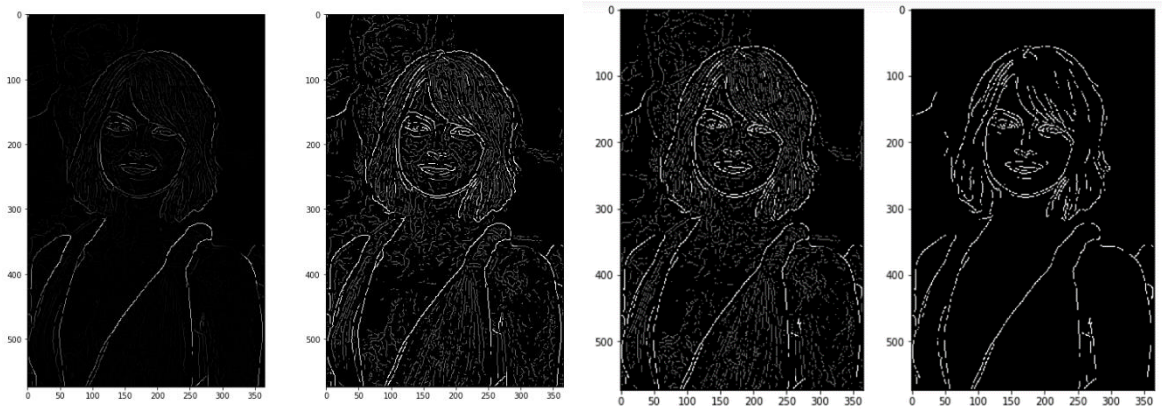


- 4. Determine edge (Double, Hysteresis threshold)

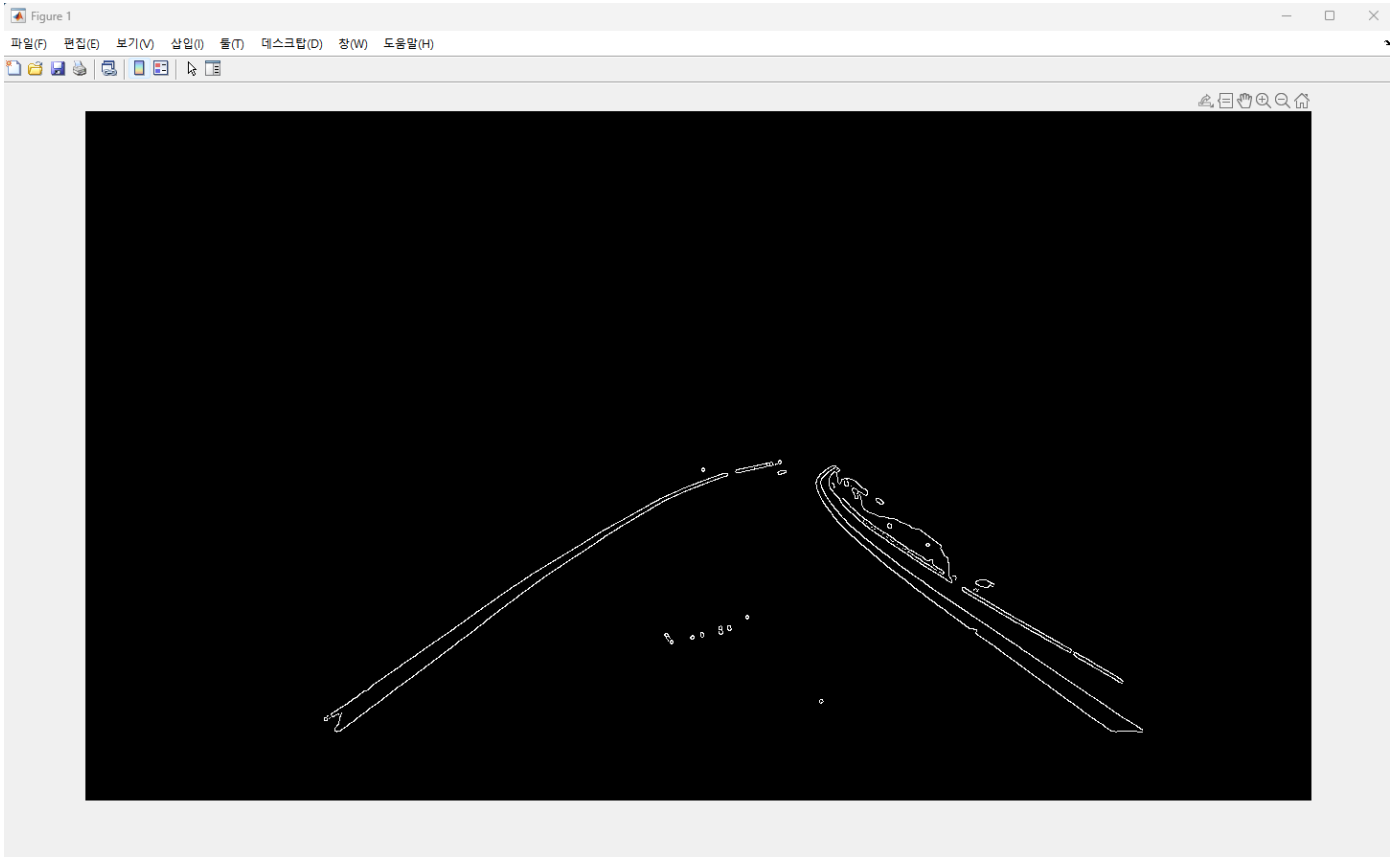
강 Edge : 유지
약 Edge : Hysteresis threshold 적용
그 외 제거





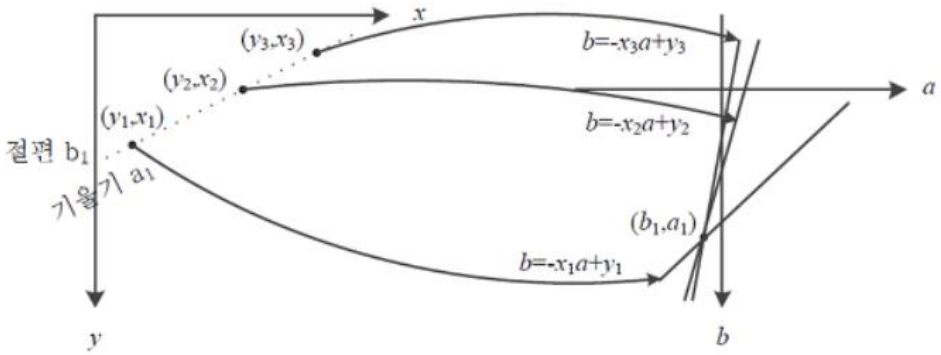


CANNY EDGE DETECTION



Canny edge를 이용한 엣지 검출.

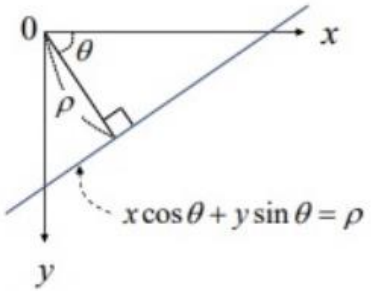
HOUGH TRANSFORM



xy평면
기울기 a1, y절편 b1인 직선 Y = a1x + b1
임의의 점 (y1, x1), (y2, x2), (y3, x3)

ab평면 (기울기, y절편의 평면)
각각 하나의 직선 → 총 3개의 직선
xy평면 상에서 같은 직선의 점 → 같은 기울기, y절편.
교점 (a1,b1) 형성.

$$x\cos\theta + y\sin\theta = \rho$$

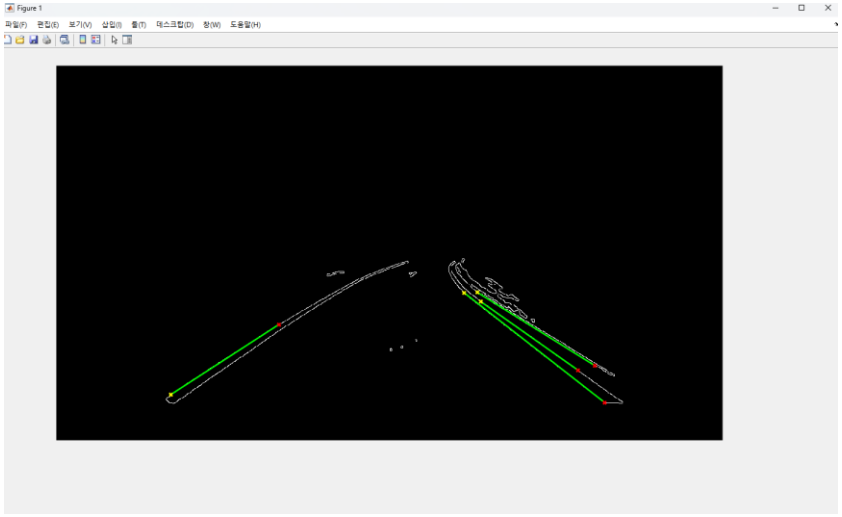


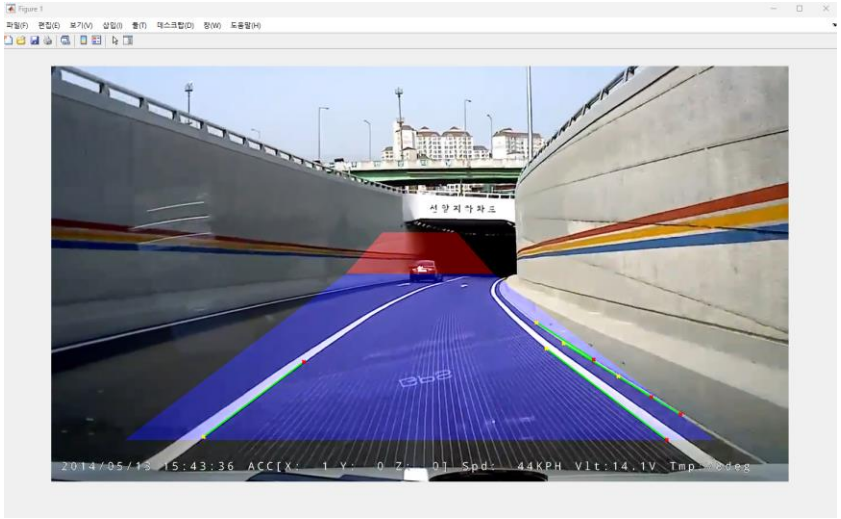
$$\begin{cases} \text{기울기} = -\dfrac{\cos\theta}{\sin\theta} \\ \text{y절편} = \dfrac{\rho}{\sin\theta} \end{cases}$$

$$y = -\frac{\cos\theta}{\sin\theta}x + \frac{\rho}{\sin\theta}$$

→ $x\cos\theta + y\sin\theta = \rho$

CANNY EDGE DETECTION





SENSOR ANALYSIS – WEEKLY SEMINAR

PRESENTER : HAKSEON LEE

ADVISE PROFESSOR : JIN WOO SONG

2024. 8.21

CONTENTS

- Rosbag
- IMU
- Odometry
- Radar
- Scheduled to proceed

ROSBAG

1x1 BagSelection

속성	값
FilePath	'/Users/leehakseon/Documents/MATLAB/Examples/R2023a/2024-05-16-15-12-15.bag'
StartTime	1.7158e+09
EndTime	1.7158e+09
NumMessages	55750
AvailableTopics	*5x3 table*
AvailableFrames	*0x1 cell*
MessageList	*55750x4 table*

- **Filepath**: The path to the rosbag file - A rosbag file is a file format used in ROS to record sensor data, messages, and topics.
  Messages: In ROS, a message is a data packet with a specific structure.
  - **Sensor Data Messages**: Data collected by various sensors, including image data, distance measurements, acceleration values, etc.
  - **Status Messages**: Information such as the robot's current location, speed, battery status, etc.
  - **Command Messages**: Specific commands, such as movement commands for the robot or commands to open/close a gripper.
- **StartTime, EndTime**: The timestamps of the first and last recorded messages, represented in UNIX time format.
- **NumMessages**: The total number of messages contained in the rosbag file.
- **AvailableTopics**: The topics recorded in the rosbag file - 5 topics, 3 attributes.
- **Imu, Lidar, Odometry, Radar, Camera**: Five types of sensors.
- **AvailableFrames**: The frames available in the rosbag - indicated as "0x1 cell," meaning the rosbag file does not contain frame information.
- **MessageList**: A label where information about all messages is stored.

IMU

msgStructs_IMU{1, 1}

속성	값
MessageType	'sensor_msgs/Imu'
Header	*1x1 Header*
Orientation	*1x1 Quaternion*
AngularVelocity	*1x1 Vector3*
LinearAcceleration	*1x1 Vector3*
OrientationCovariance	[2.7921e-05;0;0;0;2.7974e-05;0;0;0;0.0030]
AngularVelocityCovariance	[0;0;0;0;0;0;0;0;0]
LinearAccelerationCovaria...	[0;0;0;0;0;0;0;0;0]

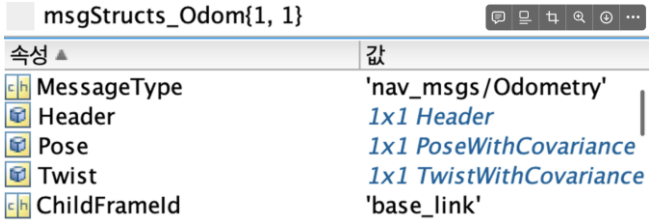
Quaternion: A mathematical method for representing rotation in 3D space, which can be converted into Euler angles, rotation matrices, and other forms.
q = w + xi + yj + zk:

w represents the scalar part (real part).

x, y, z represent the vector part (imaginary part).

- **MessageType**: 'sensor_msgs/Imu'
  This indicates that the message type is related to sensor data, specifically an IMU message, which is used to convey information about the inertial measurement unit's data in ROS.
- **Header**: 1x1 Header
  This field contains metadata about the message, such as the timestamp and frame ID. It's represented as a 1x1 structure, meaning it likely contains a single instance of a Header.
- **Orientation**: 1x1 Quaternion - X : -0.0068, Y : -0.0045, Z : 0.1399, W : 0.9901
  This field stores the orientation of the IMU as a quaternion, which is a common way to represent rotations in three-dimensional space. The 1x1 indicates a single quaternion instance.
- **AngularVelocity**: 1x1 Vector3 – X : 0.0019, Y : 0.0016, Z : -0.0011
  This field holds the angular velocity data of the IMU in three dimensions, using a vector of three components (x, y, z). It is slowly rotating clockwise around the X and Y axes (0.0019, 0.0016) and counterclockwise around the Z axis (-0.0011).
- **LinearAcceleration**: 1x1 Vector3 - X : -0.0790, Y : 0.1780, Z : -9.7918
  This field contains the linear acceleration data of the IMU in three dimensions, similar to the angular velocity, It is primarily moving in the Y-axis direction, with very small accelerations in the X and Y axes, indicating that the robot is moving in these directions with little change. The large negative acceleration in the Z-axis suggests that the robot is accelerating downward due to gravity.
- **OrientationCovariance**: [2.7921e-05;0;0;0;2.7974e-05;0;0;0;0.0030]
  This is a 3x3 matrix (flattened into a single row) that represents the covariance of the orientation data. The values provided indicate the uncertainty associated with each component of the orientation.
- **AngularVelocityCovariance**: [0;0;0;0;0;0;0;0;0]
  This is a 3x3 covariance matrix for the angular velocity data, where all values are zero, suggesting no uncertainty (or it hasn't been measured).
- **LinearAccelerationCovariance**: [0;0;0;0;0;0;0;0;0]
  Similar to the angular velocity covariance.

ODOMETRY



- Estimating "relative position" to determine how far the robot is from its starting point: The distance is measured using encoders attached to the motors (wheels). By analyzing the encoder values and the distance traveled, the robot's position is estimated using the wheel angles and the distance covered.
- **MessageType**: 'nav_msgs/Odometry'
  This indicates that the message is of type Odometry, which is used in ROS to describe the position and velocity of a robot in a given frame.
- **Header**: 1x1 Header
  This field contains metadata about the message, such as the timestamp and frame ID. The 1x1 indicates it is a single instance of a Header structure.
- **Pose**: 1x1 PoseWithCovariance
  This field holds the position and orientation of the robot. The robot's position (3D coordinates) and orientation (in Quaternion format) are included, along with a covariance matrix that represents the uncertainties associated with these values.
- **Twist**: 1x1 TwistWithCovariance
  It includes linear velocity and angular velocity, along with a covariance matrix that represents the uncertainties associated with these values.
  Since the values for all axes of both linear velocity and angular velocity are 0, it indicates that the robot is in a stationary state. Since all values in the covariance matrix are 0, it indicates that there is no uncertainty, or that uncertainty is not being considered.
- **ChildFrameId**: 'base_link'
  This is the ID of the child coordinate frame associated with the odometry data, often referred to as base_link. It represents the frame of reference for the robot's body.

RADAR

msgStructs_radar_cen{1, 1}

속성	값
PreserveStructureOnRead	0
MessageType	'sensor_msgs/PointCloud2'
Header	*1x1 Header*
Fields	*5x1 PointField*
Height	1
Width	2
IsBigendian	0
PointStep	32
RowStep	64
Data	*64x1 uint8*
IsDense	1

- A system that uses electromagnetic waves to determine the position, speed, and direction of objects.
- It operates by transmitting electromagnetic waves through a transmitting antenna, which then bounce off objects and return as reflected waves.
- By analyzing the time it takes for the reflected waves to return, along with other characteristics of the waves, the system can detect and calculate the distance, direction, and speed of the objects.

- **PreserveStructureOnRead**: 0
  This indicates whether the structure should be preserved when reading the data. A value of 0 means that the structure is not preserved.
- **MessageType**: 'sensor_msgs/PointCloud2'
  This specifies that the message type is PointCloud2, which is commonly used in ROS to represent 3D point cloud data, such as that collected from LIDAR or radar sensors.
- **Header**: 1x1 Header
  Contains metadata about the message, such as the timestamp and frame ID. The 1x1 indicates it is a single instance of a Header structure.
- **Fields**: 5x1 PointField
  Represents the fields of the point cloud data, where each field corresponds to a specific attribute of the point cloud (e.g., x, y, z coordinates, intensity). The 5x1 indicates that there are five fields in the point cloud.
- **Height**: 1
  Indicates the height of the point cloud data. A value of 1 typically means that the point cloud is organized as a single row of points, often referred to as a "unorganized" point cloud.
- **Width**: 2
  Indicates the width of the point cloud data, which is the number of points in each row.
- **IsBigendian**: 0
  Specifies the endianness of the data. A value of 0 indicates that the data is in little-endian format.
- **PointStep**: 32
  The length of a point in bytes, indicating the size of each point in the point cloud.
- **RowStep**: 64
  The length of a row in bytes, which is the total number of bytes in each row of the point cloud.
- **Data**: 64x1 uint8
  The actual point cloud data, represented as an array of unsigned 8-bit integers (uint8). The 64x1 suggests there are 64 bytes of data in total.
- **IsDense**: 1
  Indicates whether the point cloud is dense. A value of 1 means that all points in the point cloud are valid, with no invalid (NaN) points.

RADAR - FIELDS

msgStructs_radar_cen{1, 1}.Fields(1, 1)

속성	값
MessageType	'sensor_msgs/PointField'
INT8	1
UINT8	2
INT16	3
UINT16	4
INT32	5
UINT32	6
FLOAT32	7
FLOAT64	8
Name	'x'
Offset	0
Datatype	7
Count	1

msgStructs_radar_cen{1, 1}.Fields(2, 1)

속성	값
MessageType	'sensor_msgs/PointField'
INT8	1
UINT8	2
INT16	3
UINT16	4
INT32	5
UINT32	6
FLOAT32	7
FLOAT64	8
Name	'y'
Offset	4
Datatype	7
Count	1

msgStructs_radar_cen{1, 1}.Fields(3, 1)

속성	값
MessageType	'sensor_msgs/PointField'
INT8	1
UINT8	2
INT16	3
UINT16	4
INT32	5
UINT32	6
FLOAT32	7
FLOAT64	8
Name	'z'
Offset	8
Datatype	7
Count	1

msgStructs_radar_cen{1, 1}.Fields(4, 1)

속성	값
MessageType	'sensor_msgs/PointField'
INT8	1
UINT8	2
INT16	3
UINT16	4
INT32	5
UINT32	6
FLOAT32	7
FLOAT64	8
Name	'intensity'
Offset	16
Datatype	7
Count	1

msgStructs_radar_cen{1, 1}.Fields(5, 1)

속성	값
MessageType	'sensor_msgs/PointField'
INT8	1
UINT8	2
INT16	3
UINT16	4
INT32	5
UINT32	6
FLOAT32	7
FLOAT64	8
Name	'velocity'
Offset	20
Datatype	7
Count	1

- The point cloud data consists of five fields: x, y, z, intensity, and velocity.
- **Offset**: Indicates the position, in bytes, where this field starts within each point's data. In this case, it is set to 0, meaning the X coordinate data starts from the first byte of the point cloud data.
- **Datatype**: Specifies the data type in which the point cloud data is stored. The value 7 represents the FLOAT32 type, so the X coordinate value is stored in a 32-bit floating-point format.
- **Count**: Indicates the number of elements that make up this field. Here, it is set to 1, meaning there is one X coordinate value for each point.

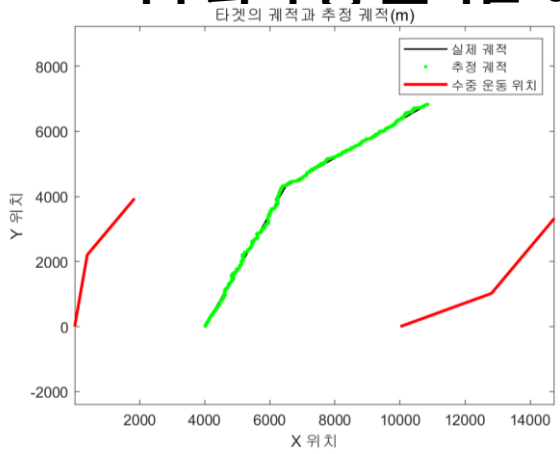
- The X coordinate is stored in a 32-bit floating-point format (FLOAT32), starting at a 0-byte offset within each point's data structure. The image provides information about the x coordinate field in 3D point cloud data.
- The Y coordinate of the 3D point cloud data is stored in a 32-bit floating-point format, and this value is located at an offset of 4 bytes after the start of each point in the array.
- The Z coordinate data is stored at an 8-byte offset position in the array, following the X and Y coordinate data. This field defines the Z coordinate of each point in the 3D point cloud data and is stored in a 32-bit floating-point format.
- The intensity data is stored at a 16-byte offset position in the array, following the X, Y, and Z coordinate data. This field defines the intensity of each point in the 3D point cloud data and is stored in a 32-bit floating-point format.
- The velocity data is stored at a 20-byte offset position in the array, following the X, Y, Z coordinate, and intensity data. This field defines the velocity of each point in the 3D point cloud data and is stored in a 32-bit floating-point format.

SCHEDULED TO PROCEED

- Plot and Visualize data of Camera, Radar, Odometry, IMU

- Paper : "Robust Object Detection and Classification Algorithm Based on LiDAR and Camera Sensor Fusion” (with 최준오, 정우주)
- Conduct literature review and presentation seminar on LiDAR and Camera.
- Increase understanding of sensors through real measurement data obtained from the WeGo Scout mini robot.
- Study algorithms for sensor fusion.
- Develop and simulate a monocular camera and LiDAR sensor fusion algorithm to verify its performance.
- Visualize sensor fusion results by displaying them on image pixel data.
- Validate sensor fusion performance through experiments using the WeGo Scout mini robot.
- Write a paper on robust object detection and classification algorithms based on LiDAR and Camera sensor fusion.

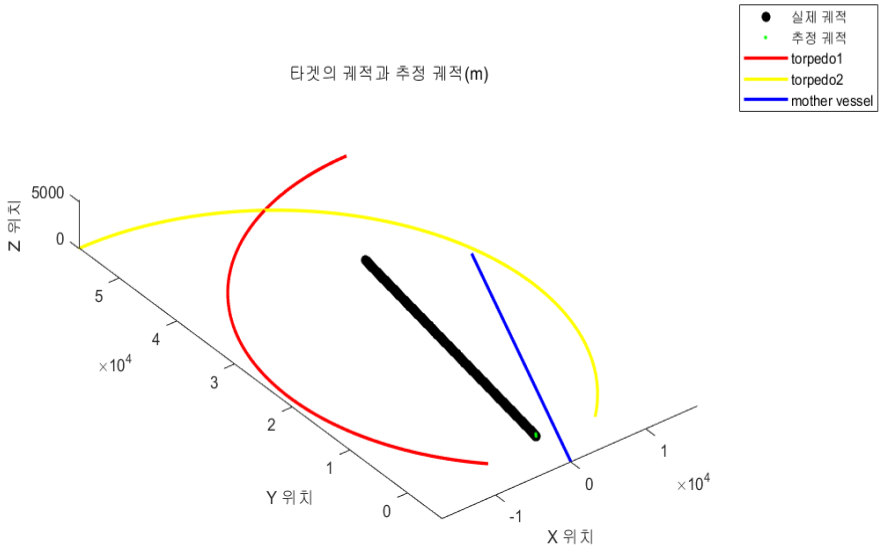
- **다수의 수동 소나를 이용한 방위각 기반 단일 표적 추적 알고리즘**

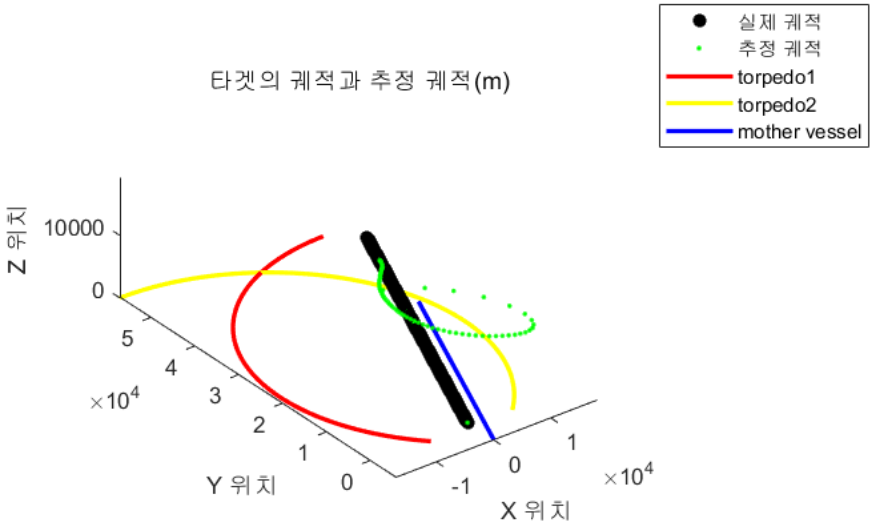


하계학술대회 우수 추천 논문 국문지 투고 요청

)에서 3D로 확장하고 어뢰 궤적에 DOP개념을 도입

간단한 시나리오를 구성





3차원 확장으로 측정값에 양각을 추가해 측정값이 2개인 EKF를 공부하고 구성하였습니다.

• 충돌시간과 dop를 고려한 어뢰의 guidance law

- 앞장과는 다르게 어뢰의 궤적이 고정되어 있지 않고 표적이 고정되어 있음
- Sea skimming 처럼 horizontal homing guidance와 vertical homing guidance로 나눔
- Horizontal homing guidance는 impact angle control guidance(충돌각제어유도법칙)사용 충돌각을 90도와 -90도로 설정해 DOP값을 내릴 수 있을 것으로 예상
- Vertical homing guidance는 Horizontal homing과 충돌시간이 같아야 하므로 Horizontal에서의 충돌시간을 계산해 PNG유도의 입력값으로 적용하는 방법으로 고민
- 예상되는 문제점은 DOP 계산식에서 유도한 것이 아니라 기하학적으로 생각 했기 때문에 DOP가 낮아지는 다른 방법이 존재할 수 있음

LiDAR Feature

Presenter : Junghwan Kim

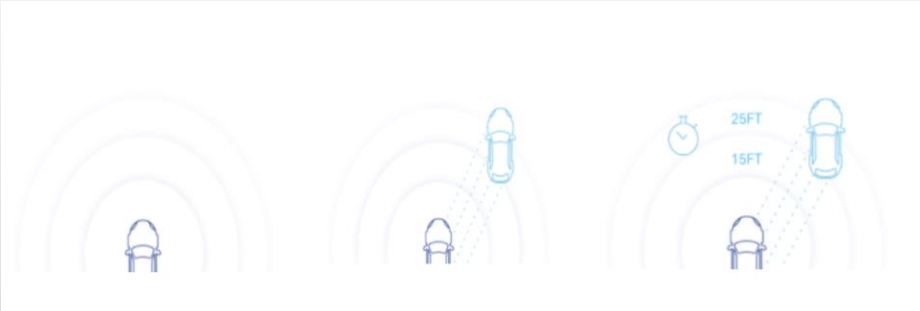
Advise Professor : Jin Woo Song

2024. 8. 21

LiDAR

Light Detection and Ranging

- 고출력 레이저 펄스를 송신
- 목표물에 맞고 수신하기 까지 **지연시간**을 분석하여 목표물까지의 거리를 측정



펄스 레이저의 파장대

- 905nm

- 소모 전력이 낮음, 양산이 쉽고 단가 낮음
- 노이즈 현상이 높음, 탐지거리가 낮음

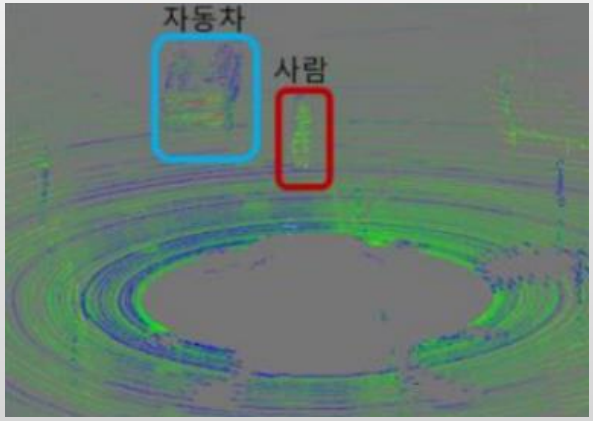
- 1550nm

- 시력 손상이 적음, 탐지거리 높음
- 단가 높음, 소모전력 높음

LiDAR

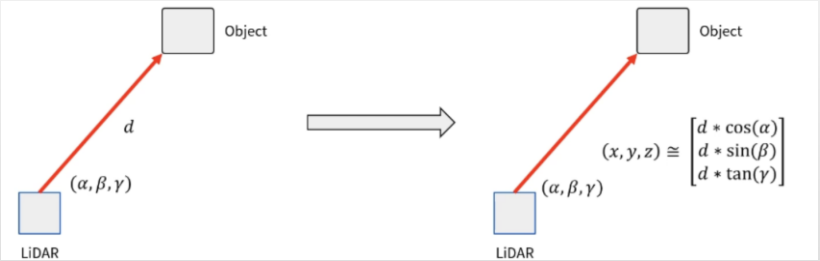
Point Cloud

- 라이다 센서가 수집한 3D 좌표 데이터의 집합
- Intensity(반사강도) : 매질의 물체에 따라 / 색상에 따라 다른 반사도를 나타냄



작동 원리

- **ToF (Time of Flight)**
  반사된 펄스 신호들이 수신기에 도착하는 **시간**을 측정 -> 거리 계산



LiDAR

회전형

- 센서가 직접 회전하며 정보를 획득

  - 360도 전방의 환경 데이터 획득이 가능
  - 구성이 복잡, 센서 가격이 높음

고정형

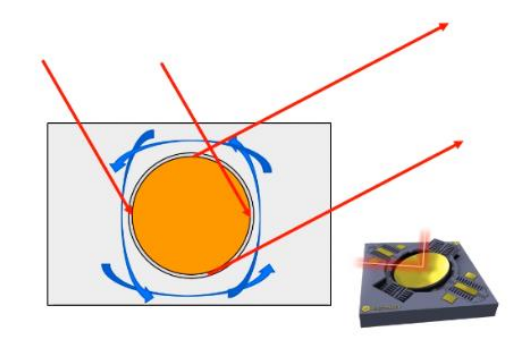
- 센서가 고정되어 한방향의 정보만 획득 가능

  - 센서 가격이 비교적 낮음
  - 제한적인 FOV를 가짐

LiDAR

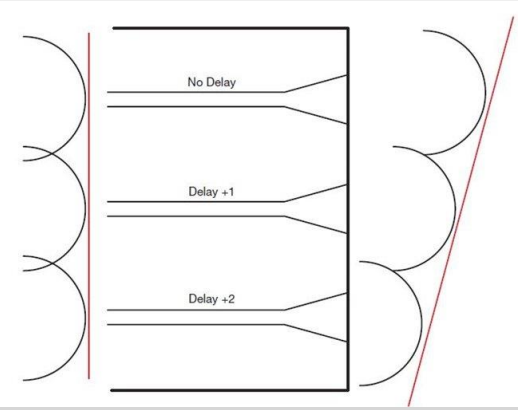
MEMS

- LiDAR내 작은 거울에 전압을 걸어 기울기 각을 변화시켜준다.
- 보다 넓은 FOV를 확보 할 수 있다.



OPA

- 빛의 간섭 원리를 이용
- 광학 위상 모듈레이터가 렌즈를 통과하는 빛의 속도를 제어



향후 계획

- LOAM 소스코드 분석
- 주변환경의 밝기 변화에 따른 차선인식

2024 Summer Research Report

Presenter : Bo Kyung Choi

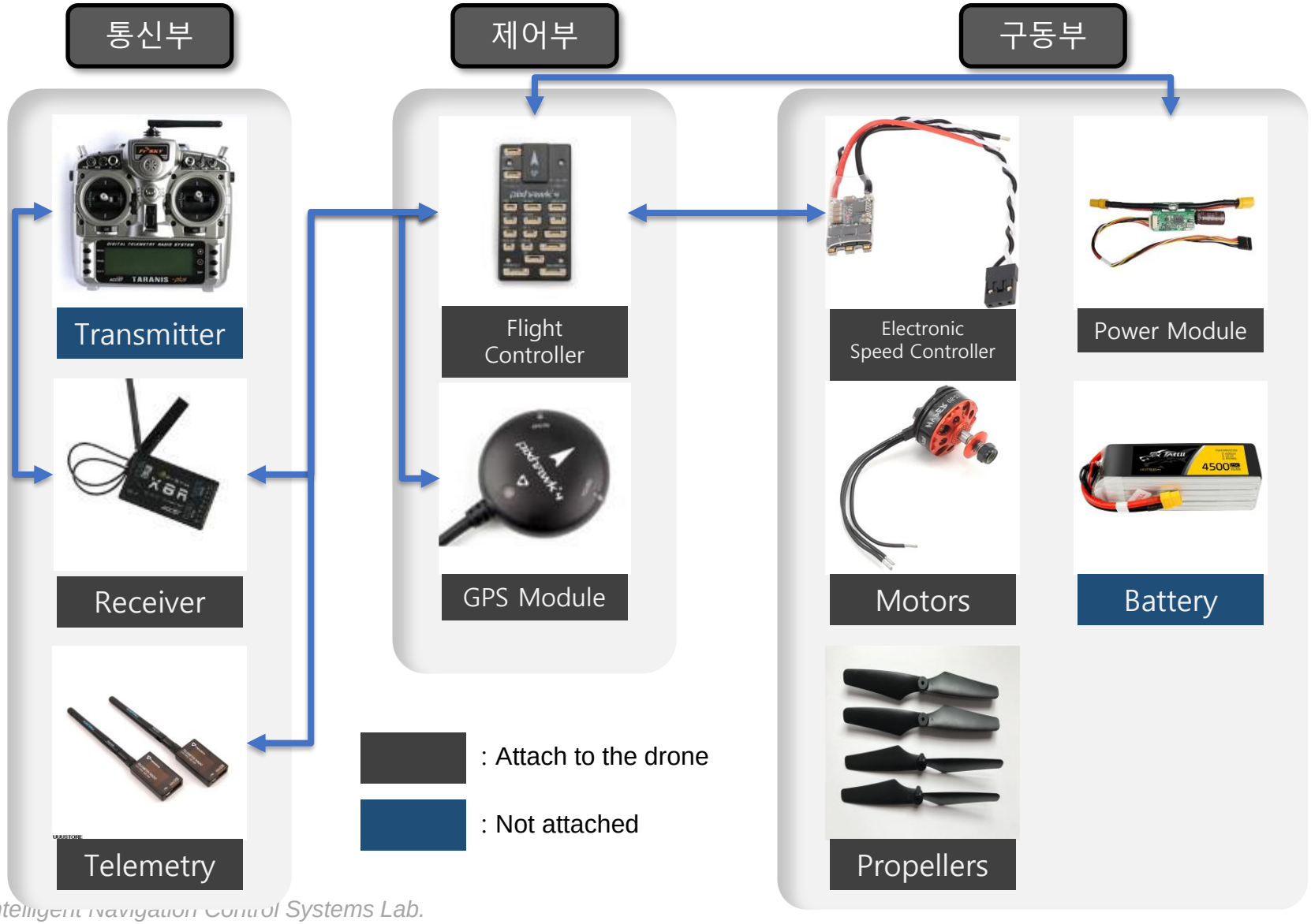
Advise Professor : Jin Woo Song

2024. 8. 21

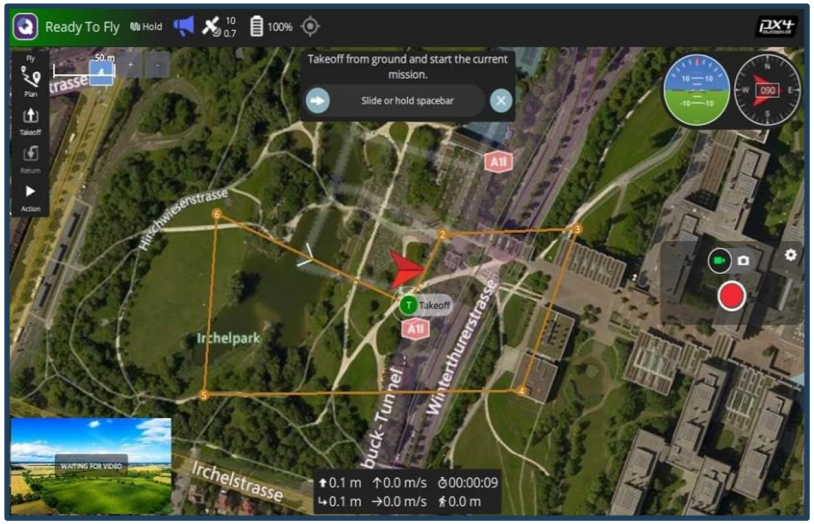
CONTENTS

- Drone - Hardware
- QGroundControl/Mission Planner
- Drone - Software

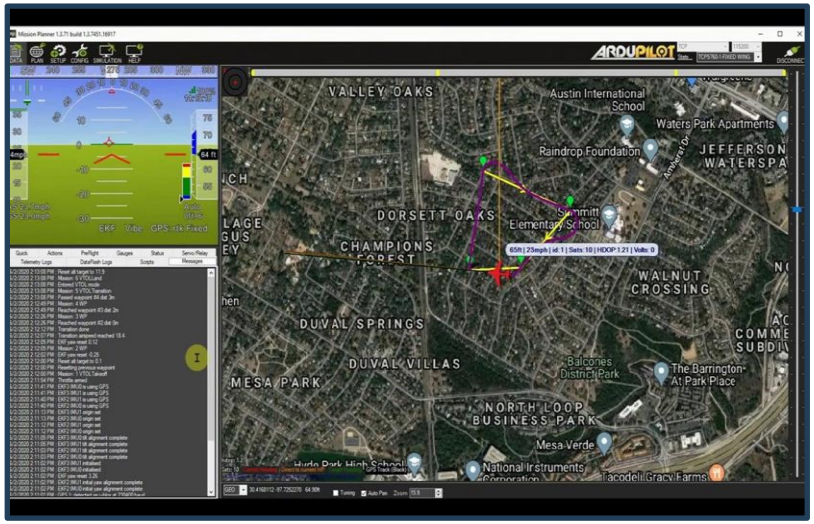
Basic Structure of Drone (Hardware)



드론 지상 제어국 (QGroundControl/Mission Planner)



QGroundControl

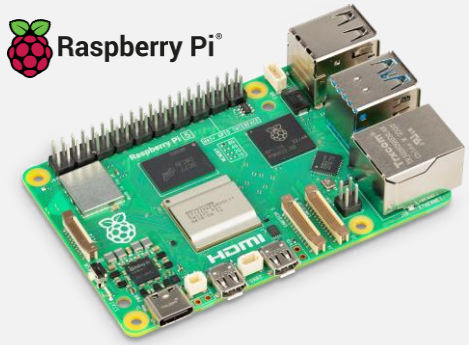


Mission Planner

- 펌웨어 로드, 펌웨어 업데이트, 센서 캘리브레이션, 라디오 설정, 플라이트 모드 설정, 파라미터 설정
- 비행 모니터링, 비행 경로 설정, 비행 시뮬레이션
- 펌웨어 종류에 따라 QGC/MP 중 선택 하여 사용

- 설치 가능한 펌웨어 종류 : PX4, ARDUPILOT

Basic Structure of Drone (Software)



Embedded Board

Ubuntu

Operating System(OS)

ROS

MAVLINK

MICRO AIR VEHICLE COMMUNICATION PROTOCOL

Middleware/Software

- 임베디드 보드 : 실시간 제어 및 데이터 처리
- 보드 OS 설치
  - 임베디드 보드 모델에 따른 OS 호환 여부 확인
- Middleware, Software 설치
  - MAVLINK : 텔레메트리를 연결한 지상국과 FC의 사이의 무선 통신을 위해 설치
  - ROS
    - 운영 체제(OS)에 따라 설치할 수 있는 ROS(Robot Operating System) 버전이 다름

연구 계획

- 2024-2학기 캡스톤 디자인
  - 드론 활용 아이디어로 진행할 계획
- 마커 활용 드론 자율 착륙 시스템

THANK YOU



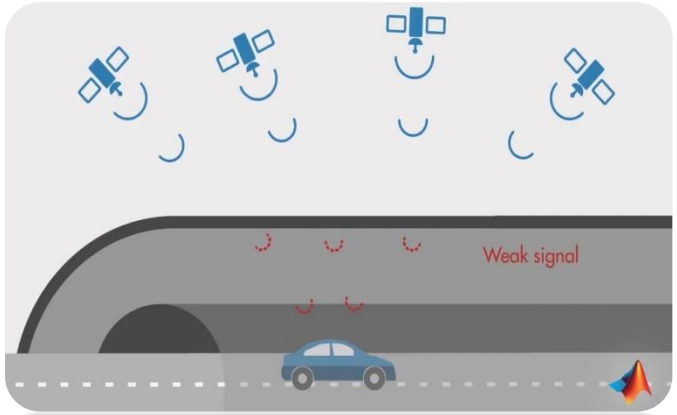
DEAD RECKONING(DR) & KALMAN FILTER

INCSL(지능형 항법 제어 시스템 연구실)
송진우 교수님 연구실

Dead Reckoning

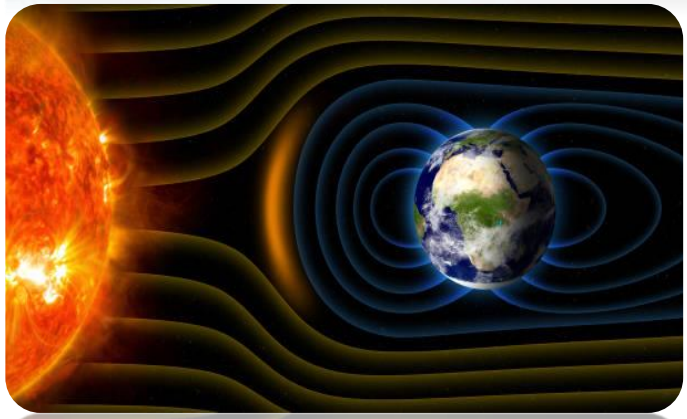
- Principles of Dead Reckoning(DR)
- Advantages and Disadvantages of Dead Reckoning (DR)

PRINCIPLES OF DEAD RECKONING(DR)

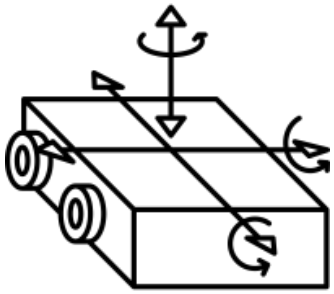




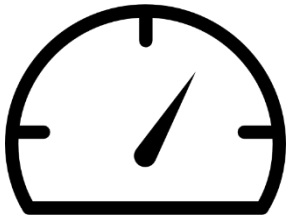




Principles of Dead Reckoning(DR)



IMU(Inertial Measurement Unit)
- Accelerometer
- Gyroscope
- Magnetometer



VSS(Vehicle Speed Sensor)



Steering Angle Sensor

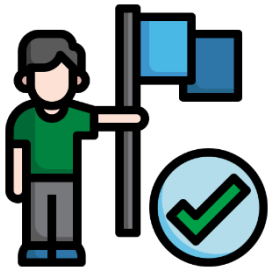
ADVANTAGES AND DISADVANTAGES OF DEAD RECKONING (DR)

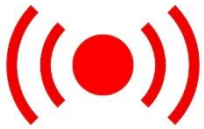
Advantages of DR

•**Independence** : Capable of measuring location autonomously without relying on GPS or other external systems.

•**Continuous Tracking** : Location information is continuously updated. this is useful in rapidly changing situations.

•**Security** : Exceptional in terms of security .





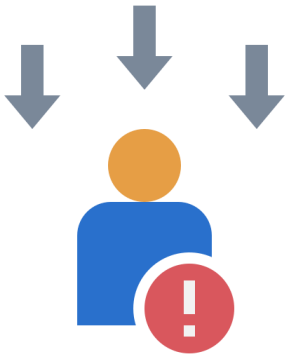


Advantages and Disadvantages of Dead Reckoning (DR)

Disadvantages of DR

- **Accumulation of Error** : Small errors can develop into significant errors as time progresses.

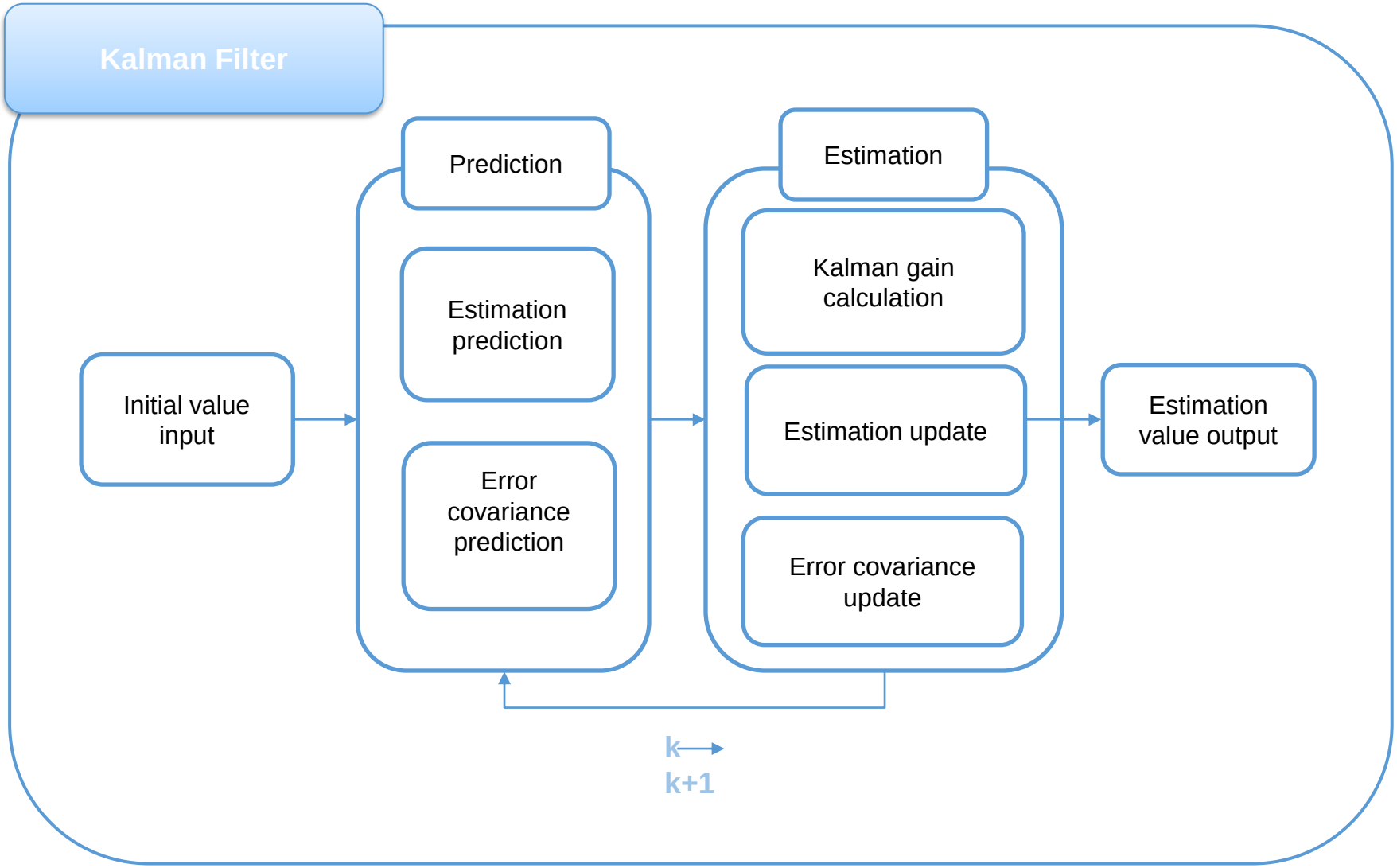
- **Accuracy** : Accuracy can be compromised by external factors such as wind, currents, and terrain.



Kalman Filter

- Principles of Kalman Filter

KALMAN FILTER



하계 활동 보고

21011942 정우주

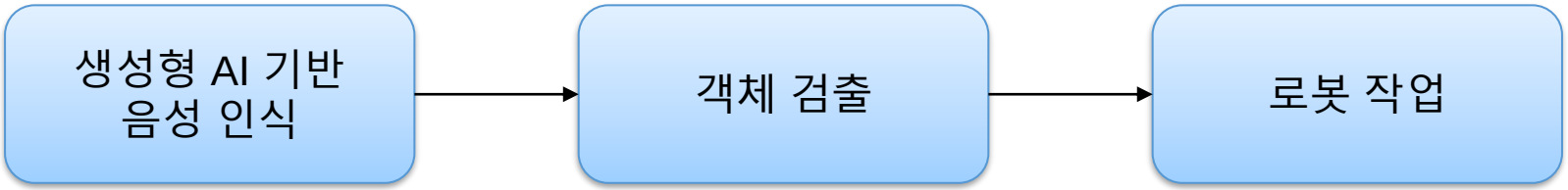
2024.08.21

CONTENTS

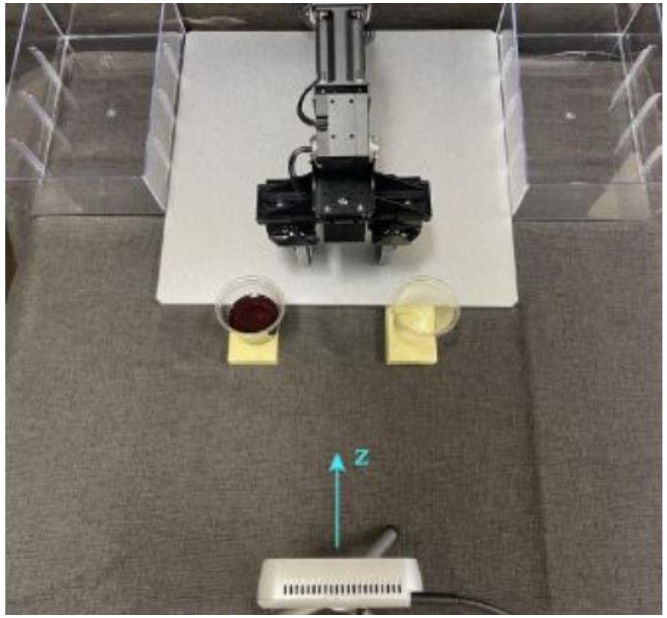
- 하계 활동
- 계획

하계 활동

1. “음성명령 및 객체 인식 기반 로봇 작업 계획” (한국로봇학회 저널), 논문 등록



- 객체 검출
  - 단안 카메라 사용
  - 데이터셋 수집 및 Yolo v4를 사용한 라벨링
  - 전이학습을 활용한 객체 검출 모델 생성
  - 객체 검출 및 정보 저장



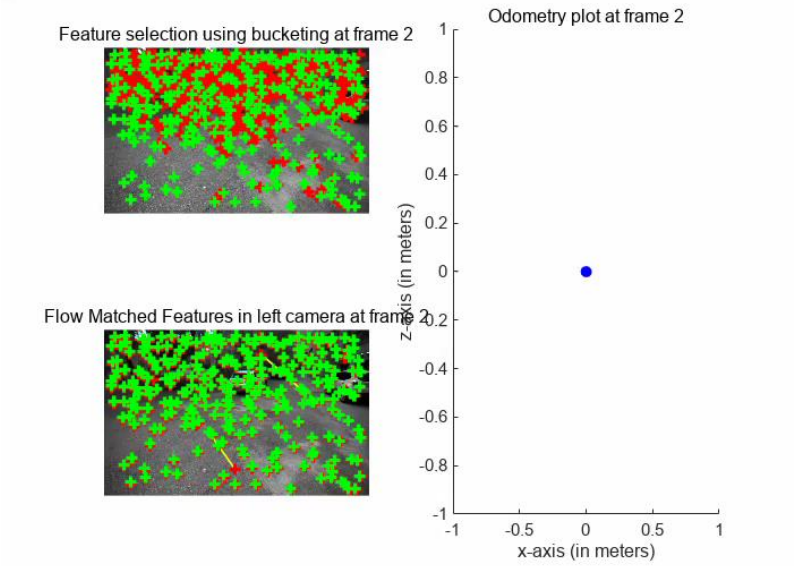
하계 활동

2. Stereo Visual Odometry

- 알고리즘

1) Feature Detection : Non-Maximum suppression 방법으로 특징점 추출
2) Feature Matching : 대응점 간 SAD 점수로 특징점 매칭, NCC 점수에 따른Outlier 제거
3) Featrue Selection : 이미지를 bucket으로 나누어 각 bucket에서 강력한 특징점만 선택
4) Egomotion Estimation : 3D 좌표계 -> 카메라 좌표계 -> 이미지 좌표계 -> R, t 추정

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f & 0 & c_u \\ 0 & f & c_v \\ 0 & 0 & 1 \end{pmatrix} \left[\begin{pmatrix} \mathbf{R}(\mathbf{r}) & \mathbf{t} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} - \begin{pmatrix} s \\ 0 \\ 0 \end{pmatrix} \right]$$



계획

1. 두런두런 활동 -> 춘계 학회 논문 작성
2. 캡스톤
3. Visual Odometry 관련 연구 진행

감사합니다



Improving performance of Monocular Visual odometry Using Depth Estimation with Size Information in Natural Randmark

Presenter : Min Jun Choi

Advise Professor : Jin Woo Song

2024.08.21

기존 연구의 한계 분석

- 형태학 기반의 번호판 인식 시스템의 한계

-> 잘못된 영역 인식, 부정확한 bounding box의 경계



-> 이미지 직접 연산으로 인한 실행 시간 증가, 실시간성 측정 불가, 번호판이라는 한정된 객체만을 탐색 가능

개선 방안

- Object Detection을 위한 인공지능 도입 예정

-> YOLO를 통한 bounding box 검출과 Image segmentation



- 번호판 뿐만이 아닌 일반화된 객체에 대하여 객체 검출 가능
- Image Processing 시간의 단축을 통한 실시간성 측정 가능 기대
- 정확한 bounding box를 통해 객체의 크기정보를 통한 depth estimation 성능의 향상 기대

연구 방향

- Monocular Visual Odometry

Transformation matrix, homogeneous coordinate

Euler angle, Quaternion

Camera Modeling, Calibration, Distortion

Epipolar geometry

PNP, ICP Problem

Triangularization

- Natural Landmark

Object Detection(YOLO, Faster R-CNN)

Segmentation

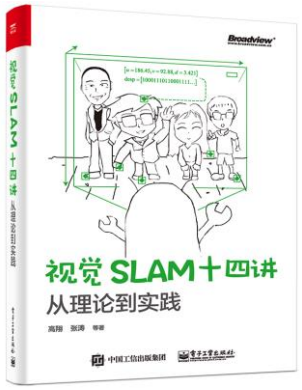


학습 진행 중

향후 계획

- "입문 Visual SLAM 이론에서 연습까지 14 강"을 통한 개념 학습 진행 중

  -> 이론 학습 후 monocular visual odometry 구현 예정



- YOLO를 통한 Natural Randmark의 bounding box 추출

- Natural Randmark의 Size Information을 통한 Depth Estimation

최종 목표

- Natural Randmark의 Size Information을 통한 Depth Estimation 과 그를 통한 Monocular Visual Odometry의 성능 개선

PROBABILITY ROBOTICS & ROS1 TUTORIALS

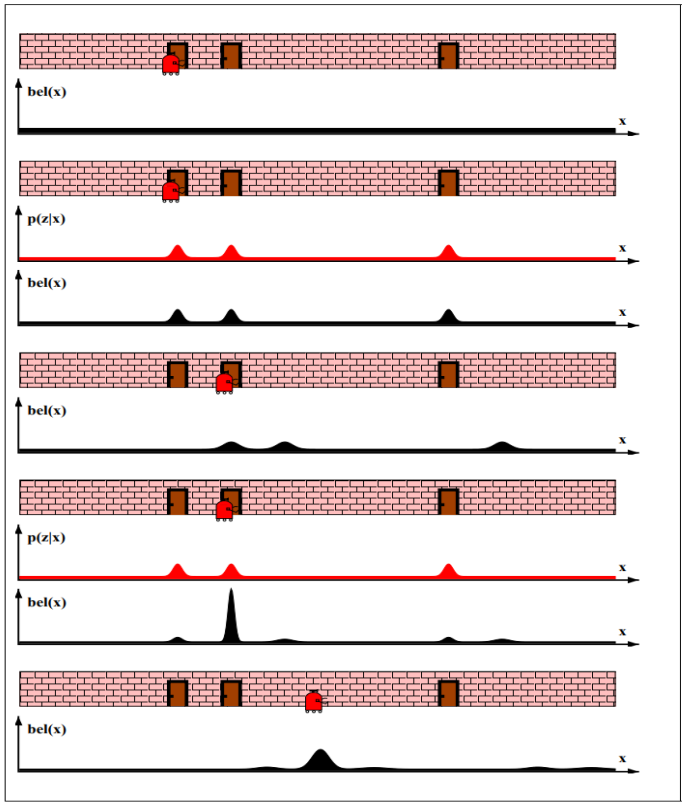
PRESENTER : HYUN JUN KIM

ADVISE PROFESSOR : JIN WOO SONG

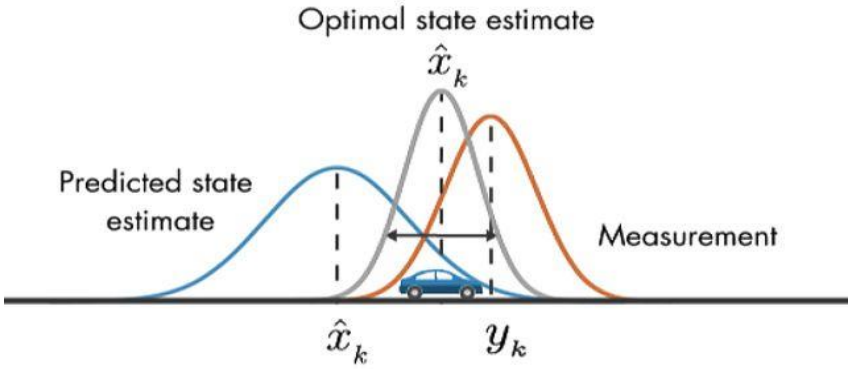
2024. 8. 21

Probability Robotics

Markov Localization



Kalman filter



Time Update ("Predict")

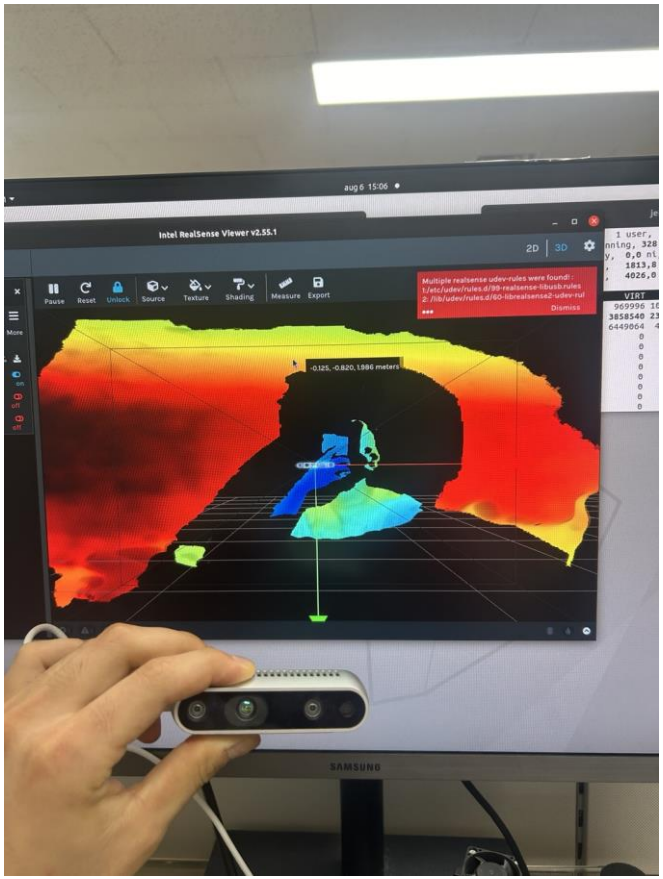
1. Extrapolate the state
   $\hat{\boldsymbol{x}}_{n+1,n} = \boldsymbol{F}\hat{\boldsymbol{x}}_{n,n} + \boldsymbol{G}\boldsymbol{u}_n$
2. Extrapolate uncertainty
   $\boldsymbol{P}_{n+1,n} = \boldsymbol{F}\boldsymbol{P}_{n,n}\boldsymbol{F}^T + \boldsymbol{Q}$

Measurement Update ("Correct")

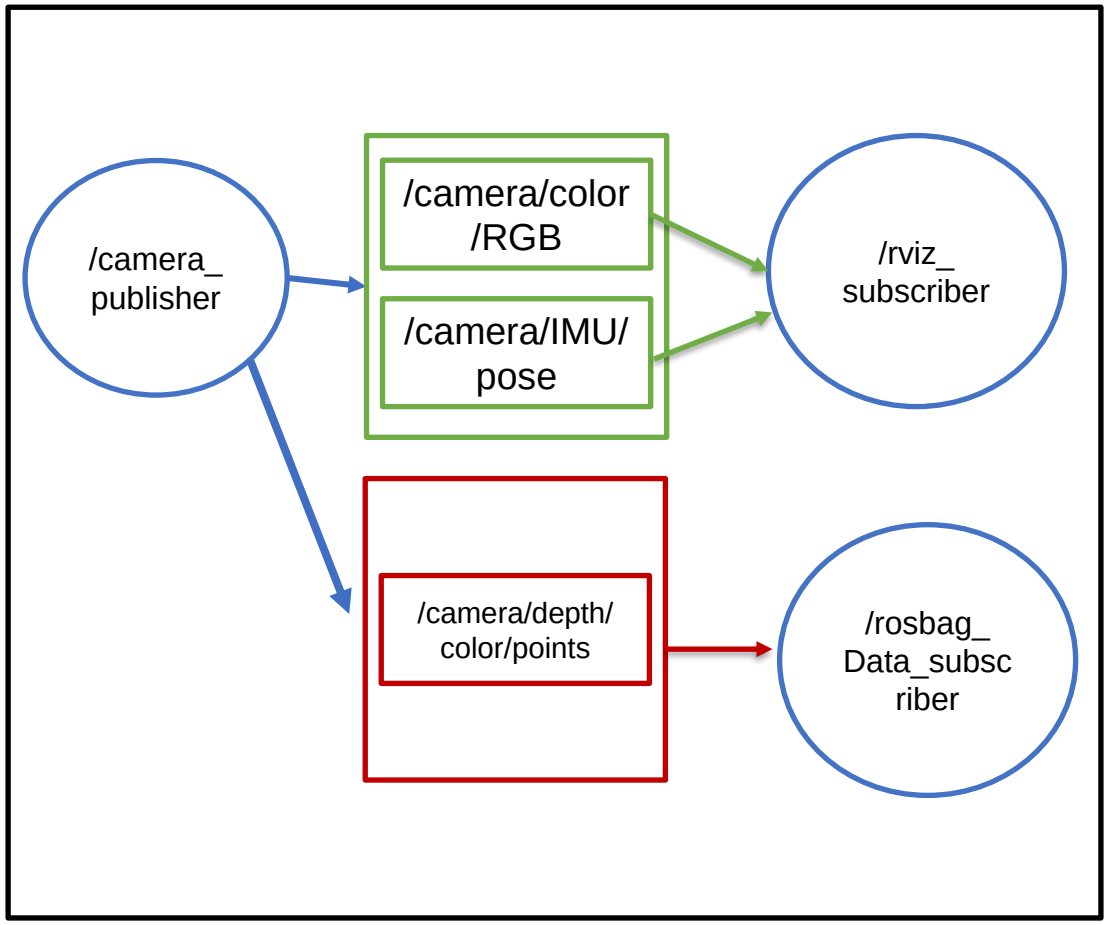
1. Compute the Kalman Gain
   $\boldsymbol{K}_n = \boldsymbol{P}_{n,n-1}\boldsymbol{H}^T\left(\boldsymbol{H}\boldsymbol{P}_{n,n-1}\boldsymbol{H}^T + \boldsymbol{R}_n\right)^{-1}$
2. Update estimate with measurement
   $\hat{\boldsymbol{x}}_{n,n} = \hat{\boldsymbol{x}}_{n,n-1} + \boldsymbol{K}_n\left(\boldsymbol{z}_n - \boldsymbol{H}\hat{\boldsymbol{x}}_{n,n-1}\right)$
3. Update the estimate uncertainty
   $\boldsymbol{P}_{n,n} = (\boldsymbol{I} - \boldsymbol{K}_n\boldsymbol{H})\boldsymbol{P}_{n,n-1}(\boldsymbol{I} - \boldsymbol{K}_n\boldsymbol{H})^T + \boldsymbol{K}_n\boldsymbol{R}_n\boldsymbol{K}_n^T$

Initial Estimate: $\hat{\boldsymbol{x}}_{0,0}$, $\boldsymbol{P}_{0,0}$

ROS1 (D435I)



RQT_GRAPH



Research of second Semester

Present ~ 10/4
1. 10/4 conference paper submit
-> Lane Detection for using Adapative, expansion ROI

10/4 ~ 12/22
2. Study 'VINS' & KALMAN FILTER using matlab
2-1. theory study of VINS & KALMAN FILTER
2-2. OPENVINS testing of using opendata